

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - CƠ - TIN HỌC



BÁO CÁO BÀI TẬP TUẦN 6
MÔN: THỊ GIÁC MÁY TÍNH (MAT3562E 2)
LAB 06: PHÂN ĐOẠN ẢNH
DỰA TRÊN VÙNG (REGION-BASED SEGMENTATION)

Giảng viên: TS. Cao Văn Chung
Sinh viên: Bùi Quang Chiến
MSSV: 23001837
Lớp: MAT3562E 2

Hà Nội, tháng 3 năm 2026

Mục lục

1	Giới thiệu	2
2	Cơ sở lý thuyết	2
2.1	Thuật toán K-means Clustering	2
2.2	Thuật toán Region Growing	3
3	Cài đặt thuật toán	3
3.1	K-means bằng NumPy — chỉ cường độ sáng	3
3.2	K-means mở rộng — cường độ + tọa độ	4
3.3	K-means bằng OpenCV	4
3.4	Region Growing — Seed Reference	5
3.5	Region Growing — Neighbor Reference (Region Mean)	5
4	Kết quả thí nghiệm	5
4.1	K1: Ảnh hưởng của số cụm k	5
4.2	K2: Ảnh hưởng của trọng số không gian	6
4.3	K3: So sánh cài đặt NumPy và OpenCV	6
4.4	R1: Region Growing — ảnh hưởng của threshold	6
4.5	R2: So sánh Seed Reference và Region Mean	6
4.6	R3: Ảnh hưởng của connectivity	6
5	Kết luận	7

1 Giới thiệu

Bài thực hành Lab 06 tập trung vào **phân đoạn ảnh dựa trên vùng** (Region-based Segmentation) — một kỹ thuật quan trọng trong thị giác máy tính. Mục tiêu là phân chia một ảnh thành các vùng có tính chất đồng nhất (cường độ sáng, màu sắc, kết cấu) nhằm phục vụ cho các tác vụ phân tích ảnh tiếp theo.

Hai phương pháp chính được nghiên cứu:

- **K-means Clustering**: Phân cụm các điểm ảnh dựa trên đặc trưng (cường độ sáng, tọa độ không gian).
- **Region Growing**: Phát triển vùng từ điểm hạt giống (seed) dựa trên tiêu chí đồng nhất.

Bài thực hành kết nối các kiến thức nền tảng đã học: histogram, xử lý điểm ảnh, phép toán số học/logic/morphology, lọc ảnh và phát hiện biên — để xây dựng pipeline phân đoạn hoàn chỉnh.

2 Cơ sở lý thuyết

2.1 Thuật toán K-means Clustering

K-means là thuật toán phân cụm không giám sát, chia tập dữ liệu thành k cụm sao cho tổng bình phương khoảng cách từ mỗi điểm đến tâm cụm gần nhất là nhỏ nhất.

Bài toán tối ưu:

$$\min_{C_1, \dots, C_k} \sum_{j=1}^k \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \mu_j\|^2$$

trong đó μ_j là tâm (centroid) của cụm C_j .

Quy trình lặp:

1. **Khởi tạo**: Chọn ngẫu nhiên k tâm cụm $\mu_1, \dots, \mu_k$.
2. **Gán nhãn**: Gán mỗi điểm $\mathbf{x}$ vào cụm có tâm gần nhất:

$$\text{label}(\mathbf{x}) = \arg \min_j \|\mathbf{x} - \mu_j\|^2$$

3. **Cập nhật tâm**: Tính lại tâm cụm:

$$\mu_j = \frac{1}{|C_j|} \sum_{\mathbf{x} \in C_j} \mathbf{x}$$

4. **Kiểm tra hội tụ**: Lặp lại bước 2–3 cho đến khi tâm cụm không thay đổi đáng kể ($\|\Delta\mu\| < \epsilon$).

Mở rộng với đặc trưng không gian: Thay vì chỉ sử dụng cường độ sáng I , ta xây dựng vector đặc trưng kết hợp:

$$\mathbf{x} = (I, w \cdot y/H, w \cdot x/W)$$

trong đó w là trọng số không gian (`spatial_weight`), giúp K-means tạo ra các vùng liền mạch hơn.

2.2 Thuật toán Region Growing

Region Growing phát triển vùng phân đoạn từ một hoặc nhiều điểm hạt giống (seed) bằng cách kiểm tra và bổ sung các điểm lân cận thỏa mãn tiêu chí đồng nhất.

Hai biến thể:

1. **Seed Reference (R1):** So sánh cường độ mỗi điểm lân cận với cường độ tại seed ban đầu:

$$|I(x, y) - I(\text{seed})| \leq T$$

Biến thể này ổn định nhưng không thích ứng tốt với gradient nội vùng.

2. **Neighbor Reference (R2):** So sánh cường độ mỗi điểm lân cận với cường độ trung bình hiện tại của vùng đã phát triển:

$$|I(x, y) - \bar{I}_{\text{region}}| \leq T$$

Biến thể này thích ứng tốt hơn với gradient nhẹ nhưng dễ bị “rò rỉ” qua biên.

Kết nối (Connectivity):

- **4-connectivity:** Xét 4 láng giềng (trên, dưới, trái, phải).
- **8-connectivity:** Xét 8 láng giềng (thêm 4 láng giềng chéo).

3 Cài đặt thuật toán

3.1 K-means bằng NumPy — chỉ cường độ sáng

Hàm `kmeans_gray_intensity` cài đặt K-means hoàn toàn bằng NumPy, sử dụng vector đặc trưng 1 chiều (cường độ sáng).

```
1 def kmeans_gray_intensity(gray, k=3, max_iter=30,  
2                             tol=1e-4, seed=0):  
3     X = gray.astype(np.float32).reshape(-1, 1)  
4     rng = np.random.default_rng(seed)  
5     init_idx = rng.choice(X.shape[0], size=k,  
6                             replace=False)  
7     centers = X[init_idx].copy()  
8  
9     for _ in range(max_iter):  
10        dists = np.sum(  
11            (X[:, None, :] - centers[None, :, :]) ** 2,  
12            axis=2)  
13        labels = np.argmin(dists, axis=1)  
14  
15        new_centers = centers.copy()  
16        for c in range(k):  
17            pts = X[labels == c]  
18            if len(pts) > 0:  
19                new_centers[c] = pts.mean(axis=0)  
20  
21        shift = np.linalg.norm(new_centers - centers)  
22        centers = new_centers  
23        if shift < tol:
```

```

24         break
25
26     label_map = labels.reshape(gray.shape)
27     return label_map, centers.reshape(-1)

```

Giải thích:

- Flatten ảnh thành vector $N \times 1$ (chỉ intensity).
- Khởi tạo k tâm bằng random sampling.
- Vòng lặp: tính khoảng cách $\rightarrow$ gán nhãn $\rightarrow$ cập nhật tâm $\rightarrow$ kiểm tra hội tụ.
- Trả về label_map (cùng kích thước ảnh gốc) và tâm cụm.

3.2 K-means mở rộng — cường độ + tọa độ

Hàm `kmeans_gray_intensity_xy` bổ sung tọa độ chuẩn hóa ($y/H, x/W$) vào vector đặc trưng với trọng số `spatial_weight`:

```

1 def kmeans_gray_intensity_xy(gray, k=3,
2     spatial_weight=0.25, max_iter=30,
3     tol=1e-4, seed=0):
4     h, w = gray.shape
5     yy, xx = np.mgrid[0:h, 0:w]
6     I = gray.astype(np.float32) / 255.0
7     X = np.stack([
8         I,
9         spatial_weight * (yy.astype(np.float32)
10             / max(h - 1, 1)),
11         spatial_weight * (xx.astype(np.float32)
12             / max(w - 1, 1))
13     ], axis=-1).reshape(-1, 3)
14     # ... (giống kmeans_gray_intensity)

```

Khi `spatial_weight` lớn, K-means ưu tiên gần gũi không gian, tạo ra các vùng liên mạch hơn nhưng có thể bỏ qua khác biệt cường độ.

3.3 K-means bằng OpenCV

OpenCV cung cấp hàm `cv2.kmeans` với tính năng `KMEANS_PP_CENTERS` (khởi tạo K-means++) và nhiều lần thử (`attempts`):

```

1 def kmeans_opencv_intensity(gray, k=2, attempts=10):
2     X = gray.reshape(-1, 1).astype(np.float32)
3     criteria = (cv2.TERM_CRITERIA_EPS +
4         cv2.TERM_CRITERIA_MAX_ITER, 50, 0.2)
5     compactness, labels, centers = cv2.kmeans(
6         X, k, None, criteria, attempts,
7         cv2.KMEANS_PP_CENTERS)
8     label_map = labels.reshape(gray.shape)
9     return label_map, centers.reshape(-1), compactness

```

Ưu điểm: Tối ưu hóa tốt hơn nhờ C++ backend và K-means++ initialization.

3.4 Region Growing — Seed Reference

```
1 def region_growing_seed_ref(gray, seed,
2                               threshold=10,
3                               connectivity=4):
4     h, w = gray.shape
5     visited = np.zeros((h, w), dtype=bool)
6     region = np.zeros((h, w), dtype=np.uint8)
7     ref_val = float(gray[seed])
8
9     if connectivity == 8:
10         neighbors = [(-1,-1),(-1,0),(-1,1),
11                     (0,-1),(0,1),
12                     (1,-1),(1,0),(1,1)]
13     else:
14         neighbors = [(-1,0),(1,0),(0,-1),(0,1)]
15
16     q = deque([seed])
17     visited[seed] = True
18     while q:
19         x, y = q.popleft()
20         if abs(float(gray[x, y]) - ref_val) <= threshold:
21             region[x, y] = 255
22             for dx, dy in neighbors:
23                 nx, ny = x + dx, y + dy
24                 if (0 <= nx < h and 0 <= ny < w
25                     and not visited[nx, ny]):
26                     visited[nx, ny] = True
27                     q.append((nx, ny))
28     return region
```

Thuật toán sử dụng BFS (hàng đợi deque) để duyệt các điểm lân cận. Tiêu chí: $|I(x, y) - I_{\text{seed}}| \leq T$.

3.5 Region Growing — Neighbor Reference (Region Mean)

Biến thể này cập nhật giá trị tham chiếu theo trung bình vùng hiện tại, thích ứng tốt hơn với gradient nhẹ:

```
1 def region_growing_region_mean(gray, seed,
2                               threshold=10,
3                               connectivity=4):
4     # ... tương tự seed_ref nhưng:
5     # - Duy trì region_sum và region_count
6     # - ref_val = region_sum / region_count
7     # - Cập nhật khi thêm điểm mới vào vùng
```

4 Kết quả thí nghiệm

4.1 K1: Ảnh hưởng của số cụm k

Thí nghiệm thay đổi $k \in \{2, 3, 4, 5, 6\}$ trên ảnh thực tế (house.jpg) với K-means chỉ dùng cường độ sáng.

Nhận xét:

- $k = 2$: Phân đoạn thô, chia ảnh thành nền/vật thể.

- $k = 3-4$: Cân bằng giữa chi tiết và đơn giản hóa.
- $k \geq 5$: Bắt đầu over-segment, các vùng nhỏ xuất hiện.
- Giá trị k tối ưu phụ thuộc vào ứng dụng cụ thể.

4.2 K2: Ảnh hưởng của trọng số không gian

So sánh K-means chỉ intensity với K-means dùng intensity + tọa độ ở các mức `spatial_weight` khác nhau.

Nhận xét:

- `spatial_weight = 0`: Tương đương K-means chỉ intensity, vùng có thể rời rạc.
- `spatial_weight = 0.25-0.5`: Các vùng trở nên liền mạch hơn.
- `spatial_weight ≥ 0.7`: Gần giống SLIC superpixels, ưu tiên vị trí hơn cường độ.

4.3 K3: So sánh cài đặt NumPy và OpenCV

Tiêu chí	NumPy	OpenCV	Ghi chú
Tốc độ	Chậm	Nhanh	OpenCV dùng C++ backend
Khởi tạo	Random	K-means++	OpenCV ổn định hơn
Linh hoạt	Cao	Trung bình	NumPy dễ tùy chỉnh đặc trưng
Chất lượng	Phụ thuộc seed	Ổn định	Nhiều lần thử (attempts)

4.4 R1: Region Growing — ảnh hưởng của threshold

Thí nghiệm trên ảnh tổng hợp (ellipse trên nền) với $T \in \{5, 10, 20, 40\}$:

Nhận xét:

- T nhỏ: Vùng nhỏ, nhiều ít ảnh hưởng, nhưng có thể thiếu sót.
- T lớn: Vùng lớn hơn nhưng dễ “rò rỉ” qua biên yếu.
- Cần chọn T phù hợp dựa trên mức nhiễu và tương phản.

4.5 R2: So sánh Seed Reference và Region Mean

Trên ảnh có gradient nội vùng, biến thể Region Mean (R2) phát triển vùng rộng hơn nhờ thích ứng dần, nhưng cũng dễ rò rỉ qua biên hơn so với Seed Reference (R1).

4.6 R3: Ảnh hưởng của connectivity

So sánh 4-connectivity và 8-connectivity:

- **4-connectivity**: Vùng có biên “vuông vắn” hơn, ít rò rỉ qua biên chéo.
- **8-connectivity**: Vùng tự nhiên hơn nhưng dễ rò rỉ qua các góc biên yếu.

5 Kết luận

Qua bài thực hành, các kết luận chính:

1. **K-means** là phương pháp đơn giản, không cần seed, nhưng cần biết trước k và không đảm bảo vùng liên thông. Việc bổ sung đặc trưng không gian giúp cải thiện tính liên mạch.
2. **Region Growing** tạo vùng liên thông tự nhiên, nhưng nhạy cảm với vị trí seed và ngưỡng T . Biến thể Region Mean thích ứng tốt hơn với gradient nhẹ.
3. **Tiền xử lý** (Gaussian blur, gamma correction) rất quan trọng để giảm nhiễu trước khi phân đoạn.
4. Không có phương pháp phân đoạn nào tối ưu cho mọi trường hợp — cần chọn phương pháp phù hợp với đặc điểm ảnh và yêu cầu ứng dụng.